

Vitis代码解析

此版Vitis代码主要功能是网络配置和数据传输，如果前面Vivado的数据存储正确，基本不用做修改，下面对代码中的结构和逻辑进行解释

Main.c

头文件说明

- `stdio.h`, `stdlib.h`, `string.h`: 用于标准的输入输出、内存操作和字符串操作。
- `xil_types.h`, `xparameters.h`: 用于Xilinx平台的类型定义和参数配置。
- 硬件接口相关**: 包括AXI GPIO (`axi_gpio_cfg.h`)、AXI DMA (`axi_dma.h`)、中断系统 (`sys_intr.h`) 和网络适配器 (`xadapter.h`)。
- lwIP相关**: 包括lwIP协议栈初始化 (`init.h`)、TCP和UDP协议相关函数 (`tcp_priv.h`, `tcp.h`)、IP地址处理 (`inet.h`)。
- `xil_printf.h`, `sleep.h`: 用于调试输出和延时操作。

宏定义与全局变量

```
#define DEFAULT_IP_ADDRESS "192.168.1.10"
#define DEFAULT_IP_MASK    "255.255.255.0"
#define DEFAULT_GW_ADDRESS "192.168.1.1"

#define MAX_PKT_LEN      4096    // 最大数据包长度
#define SEND_PKT_LEN     256     // 每次发送数据包长度

struct netif *netif;           // lwIP 网络接口结构
extern volatile int rx_done;   // 接收完成标志
extern u32 *rx_buffer_ptr;     // 接收数据缓冲区指针
u32 fifo_count = 0;
u8 dma_start_flag = 0;
```

说明

- 默认IP地址、子网掩码和网关**: 用于设备静态配置。
- `MAX_PKT_LEN` 和 `SEND_PKT_LEN`: 用于定义数据包的最大长度和每次发送的包大小。
- `netif`: lwIP 网络接口的结构体，用于配置网络接口。
- `rx_done`, `rx_buffer_ptr`: 用于指示数据接收是否完成，并指向接收数据的缓冲区。

主函数 `main`

```
int main(void)
{
    axi_gpio_init();           // 初始化 AXI GPIO
    axi_dma_cfg();             // 配置 AXI DMA
    Init_Intr_System(&Intc);   // 初始化 DMA 中断系统
    Setup_Intr_Exception(&Intc); // 设置中断异常
    dma_setup_intr_system(&Intc); // 配置 DMA 中断系统
```

```

twip_udp_init(); // 初始化 TWIP UDP

// 数据接收与发送
while (1)
{
    xemacif_input(netif); // 处理网络输入数据

    // 检查是否需要启动 DMA
    if ((dma_start_flag == 0))
    {
        axi_dma_start(MAX_PKT_LEN); // 启动 DMA 传输
        dma_start_flag = 1;
    }

    // 数据接收完成后进行 UDP 发送
    if (rx_done)
    {
        xil_printf("rx_done");
        for (int i = 1; i <= MAX_PKT_LEN / SEND_PKT_LEN / 4; i++)
        {
            udp_tx_data(rx_buffer_ptr, SEND_PKT_LEN * sizeof(u32));
            rx_buffer_ptr = (u32 *) 0x01400000 + 0x00000100 * i;
        }
        rx_buffer_ptr = (u32 *) 0x01400000; // 恢复原始地址
        rx_done = 0;
        dma_start_flag = 0;
    }
}
return 0;
}

```

说明

1. 硬件初始化

- `axi_gpio_init()`: 初始化AXI GPIO模块。
- `axi_dma_cfg()`: 配置AXI DMA传输设置。
- `Init_Intr_System(&Intc)`、`Setup_Intr_Exception(&Intc)`、`dma_setup_intr_system(&Intc)`: 初始化和配置中断系统。

2. 数据接收与发送

- `xemacif_input(netif)`: 从网络接口接收数据。
- `axi_dma_start(MAX_PKT_LEN)`: 启动DMA传输，接收数据从FIFO传输到DDR。
- `udp_tx_data(rx_buffer_ptr, SEND_PKT_LEN * sizeof(u32))`: 调用UDP发送数据函数，将接收到的数据从缓冲区发送出去。考虑到接收端的接收能力，分多次发送，逐步增大传输指针的地址

IP 配置与打印

```
static void print_ip(char *msg, ip_addr_t *ip)
{
    print(msg);
    xil_printf("%d.%d.%d.%d\r\n", ip4_addr1(ip), ip4_addr2(ip), ip4_addr3(ip),
ip4_addr4(ip));
}

static void print_ip_settings(ip_addr_t *ip, ip_addr_t *mask, ip_addr_t *gw)
{
    print_ip("Board IP:      ", ip);
    print_ip("Netmask :      ", mask);
    print_ip("Gateway :      ", gw);
}
```

说明

- `print_ip()`: 打印IP地址。
- `print_ip_settings()`: 打印IP地址、子网掩码和网关配置。

静态IP配置

```
static void assign_default_ip(ip_addr_t *ip, ip_addr_t *mask, ip_addr_t *gw)
{
    int err;

    xil_printf("Configuring default IP %s \r\n", DEFAULT_IP_ADDRESS);

    err = inet_aton(DEFAULT_IP_ADDRESS, ip);
    if (!err) xil_printf("Invalid default IP address: %d\r\n", err);

    err = inet_aton(DEFAULT_IP_MASK, mask);
    if (!err) xil_printf("Invalid default IP MASK: %d\r\n", err);

    err = inet_aton(DEFAULT_GW_ADDRESS, gw);
    if (!err) xil_printf("Invalid default gateway address: %d\r\n", err);
}
```

说明

- `assign_default_ip()`: 设置静态IP地址、子网掩码和网关。使用 `inet_aton()` 将字符串形式的IP地址转换为 `ip_addr_t` 类型。

lwIP UDP 初始化

```
int lwip_udp_init()
{
    unsigned char mac_ethernet_address[] = {0x00, 0x0a, 0x35, 0x00, 0x01, 0x02};
```

```

netif = &server_netif;

xil_printf("\r\n\r\n");
xil_printf("-----lwIP RAW Mode UDP Server Application-----\r\n");

lwip_init(); // 初始化 lwIP 协议栈

if (!xemac_add(netif, NULL, NULL, NULL, mac_ethernet_address,
PLATFORM_EMAC_BASEADDR))
{
    xil_printf("Error adding N/W interface\r\n");
    return -1;
}

netif_set_default(netif); // 设置默认网络接口
netif_set_up(netif);      // 启动网络接口

assign_default_ip(&(netif->ip_addr), &(netif->netmask), &(netif->gw)); // 配置IP

print_ip_settings(&(netif->ip_addr), &(netif->netmask), &(netif->gw)); // 打印IP配置

xil_printf("\r\n");

print_app_header(); // 打印应用标题
start_application(); // 启动应用

xil_printf("\r\n");
return 0;
}

```

说明

- `lwip_init()`: 初始化lwIP协议栈。
- `xemac_add()`: 配置网络接口, 添加到 `netif_list` 中。
- `netif_set_up()`: 启动网络接口。
- `assign_default_ip()`: 配置静态IP地址、子网掩码和网关。

axi_dma.c

配置 DMA 引擎

在系统启动时, 首先需要配置 AXI DMA 控制器, 以确保其在简单传输模式下运行。配置 DMA 引擎的函数是 `axi_dma_cfg`, 它使用 `XAxiDma_CfgInitialize` 初始化 DMA 控制器, 并确保 DMA 不在 Scatter-Gather 模式下工作。(前面Vivado关掉了此模式)

```

int axi_dma_cfg(void)
{
    int status;
    XAxiDma_Config *config;

```

```

rx_buffer_ptr = (u32 *) RX_BUFFER_BASE; // 定义接收缓冲区

xil_printf("\r\n--- Entering axi_dma_cfg --- \r\n");

config = XAxiDma_LookupConfig(DMA_DEV_ID);
if (!config) {
    xil_printf("No config found for %d\r\n", DMA_DEV_ID);
    return XST_FAILURE;
}

// 初始化 DMA 引擎
status = XAxiDma_CfgInitialize(&axidma, config);
if (status != XST_SUCCESS) {
    xil_printf("Initialization failed %d\r\n", status);
    return XST_FAILURE;
}

if (XAxiDma_HasSg(&axidma)) {
    xil_printf("Device configured as SG mode \r\n");
    return XST_FAILURE;
}

xil_printf("AXI DMA CFG Success\r\n");
return XST_SUCCESS;
}

```

启动 DMA 传输

在 DMA 引擎配置完成后，可以通过 `axi_dma_start` 函数启动 DMA 传输。该函数将接收缓冲区的地址传递给 DMA 控制器，并开始从外设接收数据。数据传输的同时，也会刷新数据缓存，确保数据能够被正确传输。

```

int axi_dma_start(u32 pkt_len)
{
    int status;
    u32 a;
    error = 0; // 错误标志初始化
    a = (u32)(pkt_len * sizeof(u32));
    xil_printf("%d\n", a);
    if (XAxiDma_Busy(&axidma, XAXIDMA_DEVICE_TO_DMA)) {
        xil_printf("DMA busy1\n");
    }
    status = XAxiDma_SimpleTransfer(&axidma, (u32) rx_buffer_ptr,
                                    (u32)(pkt_len * sizeof(u32)),
XAXIDMA_DEVICE_TO_DMA);
    if (status != XST_SUCCESS) {
        xil_printf("AXI DMA Start FAILURE\r\n");
        return XST_FAILURE;
    }

    xil_DCacheFlushRange((UINTPTR) rx_buffer_ptr, pkt_len); // 刷新数据缓存
    if (XAxiDma_Busy(&axidma, XAXIDMA_DEVICE_TO_DMA)) {
        xil_printf("DMA busy2\n");
    }
    return XST_SUCCESS;
}

```

```
}
```

中断处理

DMA 接收中断处理

当 DMA 完成数据传输时，会触发中断。`rx_intr_handler` 函数负责处理 DMA 中断。它会检查是否发生了传输错误或是否完成了接收。根据中断状态，程序会采取相应的操作，如重置 DMA 或标记接收完成。

```
void rx_intr_handler(void *callback)
{
    u32 irq_status;
    int timeout;
    XAxiDma *axidma_inst = (XAxiDma *) callback;

    irq_status = XAxiDma_IntrGetIrq(axidma_inst, XAXIDMA_DEVICE_TO_DMA);
    XAxiDma_IntrAckIrq(axidma_inst, irq_status, XAXIDMA_DEVICE_TO_DMA);

    // 错误处理
    if ((irq_status & XAXIDMA_IRQ_ERROR_MASK)) {
        error = 1;
        xil_printf("XAxiDma error");
        XAxiDma_Reset(axidma_inst);
        timeout = RESET_TIMEOUT_COUNTER;
        while (timeout) {
            if (XAxiDma_ResetIsDone(axidma_inst))
                break;
            timeout -= 1;
        }
        return;
    }

    // 传输完成处理
    if ((irq_status & XAXIDMA_IRQ_IOC_MASK))
        rx_done = 1;

    irq_status = XAxiDma_IntrGetIrq(axidma_inst, XAXIDMA_DEVICE_TO_DMA);
}
```

DMA 中断系统设置

为了处理 DMA 中断，需要配置中断控制器。在 `dma_setup_intr_system` 函数中，设置中断优先级和触发类型，并连接 DMA 中断处理函数。启用 DMA 中断后，当 DMA 发生中断时，将触发 `rx_intr_handler` 函数。

```
int dma_setup_intr_system(XScuGic * int_ins_ptr)
{
    int status;
    // 设置优先级和触发类型
    XScuGic_SetPriorityTriggerType(int_ins_ptr, RX_INTR_ID, 0xA0, 0x3);

    // 连接中断处理函数
    status = XScuGic_Connect(int_ins_ptr, RX_INTR_ID,
```

```
        (Xil_InterruptHandler) rx_intr_handler, &axidma);
    if (status != XST_SUCCESS) {
        return status;
    }
    XScuGic_Enable(int_ins_ptr, RX_INTR_ID);

    // 启用 DMA 中断
    XAxiDma_IntrEnable(&axidma, XAXIDMA_IRQ_ALL_MASK, XAXIDMA_DEVICE_TO_DMA);

    return XST_SUCCESS;
}
```

相关DMA函数

```
u32 XAxiDma_SimpleTransfer(XAxiDma *InstancePtr, UINTPTR BuffAddr, u32 Length, int
Direction)
```

名称	代码	解释
函数名	XAxiDma_SimpleTransfer	在系统中提交一次DMA的传输行为的申请
参数1	XAxiDma *InstancePtr	指向AxiDma驱动实例的指针
参数2	UINTPTR BuffAddr	源/目标缓存的地址
参数3	u32 Length	传输的长度（字节）
参数4	u32 Length,int Direction	XAXIDMA_DMA_TO_DEVICE //DDR到外设,通过DMA进行搬运 XAXIDMA_DEVICE_TO_DMA //外设到DDR,通过DMA进行搬运

```
void Xil_DCacheFlushRange(volatile u32 *Addr, u32 Size);
```

名称	代码	解释
函数名	Xil_DCacheFlushRange	用于刷新特定范围内的数据缓存，确保数据的一致性
参数1	volatile u32 *Addr	指向要刷新的数据缓存区域的起始地址
参数2	u32 Size	要刷新的数据缓存区域的大小（以字节为单位）

axi_gpio_cfg.c

定义和常量

```
#define AXI_GPIO_0_ID XPAR_AXI_GPIO_0_DEVICE_ID // AXI GPIO 0 设备 ID
#define AXI_GPIO_0_CHANNEL1 1
```

- `AXI_GPIO_0_ID`：定义了 AXI GPIO 0 的设备 ID。这个 ID 用于初始化 AXI GPIO 驱动程序。
- `AXI_GPIO_0_CHANNEL1`：表示将使用的通道号（1）。在设置方向或执行 GPIO 操作时需要指定这个通道号。

声明 AXI GPIO 驱动实例

```
xGpio axi_gpio_inst0; // AXI GPIO 0 驱动实例
```

创建一个 `xGpio` 类型的实例 `axi_gpio_inst0`，该实例用于保存 AXI GPIO 设备的信息和状态。

AXI GPIO 初始化

`axi_gpio_init` 函数

```
void axi_gpio_init(void)
{
    // AXI GPIO 0 驱动初始化
    XGpio_Initialize(&axi_gpio_inst0, AXI_GPIO_0_ID);
    // 设置 AXI GPIO 0 使用通道 1 作为输出
    XGpio_SetDataDirection(&axi_gpio_inst0, AXI_GPIO_0_CHANNEL1, 0);
}
```

- `XGpio_Initialize`：初始化 AXI GPIO 驱动程序，`axi_gpio_inst0` 作为实例，`AXI_GPIO_0_ID` 作为设备 ID。
- `XGpio_SetDataDirection`：设置 AXI GPIO 通道 1 的数据方向为输出。0 表示输出模式。

使用 AXI GPIO 控制输出

`axi_gpio_out1` 函数

```
void axi_gpio_out1(void)
{
    XGpio_DiscreteWrite(&axi_gpio_inst0, 1, 0x01);
}
```

- `XGpio_DiscreteWrite`：向 AXI GPIO 通道 1 写入数据 `0x01`，使 GPIO 引脚输出高电平。

`axi_gpio_out0` 函数

```
void axi_gpio_out0(void)
{
    XGpio_DiscreteWrite(&axi_gpio_inst0, 1, 0x00);
}
```

- `XGpio_DiscreteWrite`：向 AXI GPIO 通道 1 写入数据 `0x00`，使 GPIO 引脚输出低电平。

相关GPIO函数

```
int XGpio_Initialize(XGpio * InstancePtr, u16 DeviceId)
```

名称	代码	解释
函数名	XGpio_Initialize	初始化GPIO
参数1	XGpio * InstancePtr	指向GPIO实例的指针
参数2	u16 DeviceId	ID号，自动生成，在xparameters.h文件中定义
返回值	int	XST_SUCCESS/XST_FAILURE

```
void XGpio_SetDataDirection(XGpio *InstancePtr, unsigned Channel,u32 DirectionMask)
```

名称	代码	解释
函数名	XGpio_SetDataDirection	设置GPIO为输入/输出
参数1	XGpio * InstancePtr	指向GPIO实例的指针
参数2	unsigned Channel	待设置GPIO的通道（Vivado中设置gpio IP时的设置通道，为1或2）
参数3	u32 DirectionMask	方向设置。0: output; 1: input

```
u32 XGpio_DiscreteRead(XGpio * InstancePtr, unsigned Channel)
```

名称	代码	解释
函数名	XGpio_DiscreteRead	读取GPIO的值
参数1	XGpio * InstancePtr	指向GPIO实例的指针
参数2	unsigned Channel	通道号，同上一函数
返回值	u32	最多32位的实际值

```
void XGpio_DiscreteWrite(XGpio * InstancePtr, unsigned Channel, u32 Data)
```

名称	代码	解释
函数名	XGpio_DiscreteWrite	写GPIO
参数1	XGpio * InstancePtr	指向GPIO实例的指针
参数2	unsigned Channel	通道号，同上一函数
参数3	u32 Data	需要写的值

```
void XGpio_DiscreteSet(XGpio * InstancePtr, unsigned Channel, u32 Data)
```

名称	代码	解释
函数名	XGpio_DiscreteSet	置位某些GPIO的位（给某些GPIO的位写1）
参数1	XGpio * InstancePtr	指向GPIO实例的指针
参数2	unsigned Channel	通道号，同上一函数
参数3	u32 Data	需要给哪些位写1

```
void XGpio_DiscreteClear(XGpio * InstancePtr, unsigned Channel, u32 Data)
```

名称	代码	解释
函数名	XGpio_DiscreteClear	清零某些GPIO的位（给某些GPIO的位写0）
参数1	XGpio * InstancePtr	指向GPIO实例的指针
参数2	unsigned Channel	通道号，同上一函数
参数3	u32 Data	需要给哪些位写0

udp_perf_server.c

引入必要的头文件

```
#include <stdio.h>
#include <string.h>
#include "lwip/err.h"
#include "lwip/udp.h"
#include "xil_printf.h"
#include "lwip/inet.h"
```

- `stdio.h` 和 `string.h` 用于标准输入输出和字符串操作。
- `lwip/err.h` 提供了lwIP协议栈中的错误代码定义。
- `lwip/udp.h` 包含了UDP协议相关的函数和结构体。
- `xil_printf.h` 是Xilinx的打印函数，类似于标准的 `printf`，但用于嵌入式系统开发。
- `lwip/inet.h` 提供了IP地址相关的函数，如转换和表示IP地址。

变量定义

```
static struct udp_pcb *pcb;
#define SER_PORT 8000
```

- `pcb`：指向UDP控制块的指针，UDP控制块用于管理UDP连接。

- `SER_PORT`: 定义了UDP通信所使用的端口号, 默认为8000。

打印应用标题

```
void print_app_header()
{
    xil_printf("\r\n-----Network port UDP transmission adc sample data on upper
computer ----- \r\n");
}
```

- `print_app_header` 函数用于打印应用程序的标题, 帮助用户识别和调试。

UDP数据发送函数

```
void udp_tx_data(u32 *buffer_ptr, unsigned int len) {
    static struct pbuf *ptr;

    ptr = pbuf_alloc(PBUF_TRANSPORT, len, PBUF_POOL); // 申请缓冲区

    if (ptr) {
        pbuf_take(ptr, buffer_ptr, len); // 将buffer_ptr中的数据复制到pbuf中
        udp_send(pcb, ptr);             // 发送UDP数据
        pbuf_free(ptr);                 // 释放pbuf
    }
}
```

- `udp_tx_data` 函数用于将采集到的数据通过UDP发送。首先, 它申请一个缓冲区 (`pbuf`), 然后将数据从 `buffer_ptr` 复制到 `pbuf` 中, 最后通过 `udp_send` 函数将数据发送出去。
- `pbuf_free` 函数释放 `pbuf`, 确保内存得到回收。

启动应用程序

```
void start_application()
{
    err_t err;

    /* 设置目标IP */
    ip_addr_t DestIPAddr;
    IP4_ADDR(&DestIPAddr, 192, 168, 1, 102);

    // 创建UDP控制块
    pcb = udp_new();
    if (!pcb) {
        xil_printf("Error creating PCB. Out of Memory\r\n");
        return;
    }

    // 绑定端口
    err = udp_bind(pcb, IP_ADDR_ANY, SER_PORT); // 绑定8000端口
    if (err != ERR_OK) {
        xil_printf("Unable to bind to port %d; err %d\r\n", SER_PORT, err);
        udp_remove(pcb);
        return;
    }
}
```

```

}

// 设置目标端口
udp_connect(pcb, &DestIPAddr, 8000); // 目标地址为192.168.1.102, 端口为8000

xil_printf("UDP server started @ port %d\n\r", SER_PORT);
}

```

- `start_application` 函数初始化UDP连接。首先创建一个新的UDP控制块 (`udp_new`) 并检查是否成功。
- 接着, 绑定本地端口 (8000端口), 如果绑定失败, 则打印错误信息并退出。
- 然后设置目标IP地址和端口, 通过 `udp_connect` 函数指定连接的服务器。
- 最后, 打印提示信息, 指示UDP服务器已启动。

UDP Vitis配置

但凡使用lwip的程序, 无论TCP还是UDP, 在进入while(1)循环前, 都会有这样一个配置流程:

- 设置开发板MAC地址
- 开启中断系统
- 设置本地IP地址
- 初始化lwIP
- 添加网络接口
- 设置默认网络接口
- 启动网络
- 初始化TCP或UDP连接 (自定义函数)

<https://bestfpga.blog.csdn.net/article/details/88689771?spm=1001.2014.3001.5502>

可以自行阅读本博客 (12) - (16) 的内容以对lwip udp的代码结构有更好的理解